Chapitre 0 — Avant tout | Partie E — La croix : le joueur | leçon 0.74.1 sur 153 | difficulté 1 / 10

0.74.1. Glisser plus vite : glisse_croix à 3 — de base, ailleurs

Le 0.74 avec le B, parti de (20, 20) pixels.

1. Ce qu'il faut comprendre

C'est le 0.74, avec le **B**, parti de **(20, 20)** pixels.

C'est la version de base : la deuxième place, seule. Le 0.74.2 met les deux ensemble.

> **Les fonctions de cette leçon — et le tuto de chacune**
>
> - `#include <glisse_croix>` → 0.70.3. La fonction glisse_croix() — suivre la croix, au pixel près
> - `#include <ALPHABET>` → 0.1.4. ALPHABET — les lettres de la police

📖 Les mots de cette leçon — si tu débutes

- **accolades { }** — Elles entourent un bloc : les lignes qui vont ensemble (le corps d'une fonction, d'une boucle, d'un « if »). Chaque « { » a son « } ».
- **affectation (=)** — « x = 5; » : la boîte x prend la valeur 5. Ce n'est pas une égalité : c'est un rangement.
- **argument** — Une valeur donnée à une fonction quand on l'appelle, entre ses parenthèses : dans texte(5, 8, "SALUT"), 5, 8 et "SALUT" sont les arguments.
- **bloc** — Un groupe de lignes entre accolades { }, qui s'exécutent ensemble.
- **booléen (vrai / faux)** — Le résultat d'une condition : vrai (1) ou faux (0). « true » veut dire vrai, « false » faux.
- **boucle** — Des instructions qui se répètent : « while » (tant que…) ou « for » (pour un compteur qui va de… à…).
- **boucle du jeu** — « while (true) { image(); … } » : ce qui se répète 60 fois par seconde, sans fin, pendant tout le jeu.
- **commentaire (//)** — Tout ce qui suit « // » jusqu'au bout de la ligne : une phrase pour l'humain, que la console ignore.
- **condition** — Une question dont la réponse est vrai ou faux : « score > 10 », « bouton(A) ». On la teste avec « if » ou « while ».
- **image()** — Attend l'image suivante. La console en montre 60 par seconde : c'est ce qui règle la vitesse d'un jeu.
- **#include** — En haut du programme : « #include <texte> » demande la fonction texte() à la console ; « #include "fichier.cpp" » verse un de tes fichiers à cet endroit.
- **indice** — Le numéro d'une case dans un tableau, entre crochets : cases[3] est la quatrième case (on compte à partir de 0).
- **instruction** — Un ordre donné à la console, terminé par un point-virgule : « texte(1, 1, "OK"); ».
- **main** — « int main() { … } » : là où le programme commence. La console exécute d'abord la première ligne de main.
- **octet** — Un nombre de 0 à 255 : la plus petite boîte de la mémoire. Une cartouche en a 32 768.
- **point-virgule (;)** — Il termine une instruction, comme le point termine une phrase.
- **uint8_t** — Le genre de nombre de cette console : un entier de 0 à 255 (un octet). Au-delà, il repart à 0.
- **variable** — Une boîte avec un nom, dans la mémoire, qui retient un nombre qu'on peut lire et changer.
- **while (tant que)** — Répète un bloc tant qu'une condition est vraie.

🔍 Le programme, ligne par ligne — chaque ligne dite en français

`#include <glisse_croix>  // un lutin qui suit la croix, au pixel près`	Demande à la console « glisse_croix » (ce que c'est : un lutin qui suit la croix, au pixel près). Sans cette ligne, le compilateur refuserait de l'employer. (Le commentaire le dit : « un lutin qui suit la croix, au pixel près ».)
`#include <ALPHABET>  // les lettres de la police : ALPHABET[0] est le A`	Demande à la console « ALPHABET » (ce que c'est : les lettres de la police : ALPHABET[0] est le A). Sans cette ligne, le compilateur refuserait de l'employer. (Le commentaire le dit : « les lettres de la police : ALPHABET[0] est le A ».)
`uint8_t px = 20;  // le B, en pixels`	Une variable : une boîte appelée px, qui retient un nombre de 0 à 255, et qui commence à 20. (Le commentaire le dit : « le B, en pixels ».)

`uint8_t py = 20;`	Une variable : une boîte appelée py, qui retient un nombre de 0 à 255, et qui commence à 20.
`int main() {`	Le début du programme : la console commence ici, par la première ligne qui suit.
`while (true) {`	La boucle du jeu : tout ce qui est entre les accolades recommence sans fin, 60 fois par seconde avec image().
`image();`	Attendre l'image suivante : la console en montre 60 par seconde. C'est ce qui règle la vitesse du jeu.
`glisse_croix(0, px, py, ALPHABET[1], 3);`	Appel de la fonction glisse_croix de la console, avec les arguments 0, px, py, ALPHABET[1], 3. Ce qu'elle fait : la même chose au pixel près, avec un lutin. vitesse : pixels par image (1 = 60 pixels par seconde, 3 = 180) ; elle peut être une variable. px de 0 à 152, py de 0 à 136. Seule sur sa ligne : glisse_croix(0, px, py, ALPHABET[0], 1); range la place dans px et py.
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1142 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

```
Gravé dans la cartouche :
  démarrage            62 o  ← la console, à l'allumage  — la mise en route de la console
  glisse_croix()      195 o  ← main()  — une fonction fournie par le langage
  main()               38 o  ← démarrage  — ta fonction
  VBlank                8 o  ← la console, à chaque image  — compte les images, 60 fois par seconde
  AttendreImage        28 o  ← image()  — attend l'image suivante
  AttendreVBlank       16 o  ← démarrage  — attend que l'écran soit libre
  RangerLutins         11 o  ← démarrage  — range les 40 lutins hors de l'écran
  InstallerTransfert   15 o  ← démarrage  — installe la copie des lutins en mémoire rapide
  CopierTuiles         18 o  ← démarrage  — copie les lettres et les dessins en mémoire vidéo
  LireManette          37 o  ← bouton()  — lit les huit boutons
  Tuiles              704 o  ← CopierTuiles  — la police entière
  DonneesTransfert     10 o  ← InstallerTransfert  — la petite routine qui copie les lutins
Lettres gravées : toutes (sprite() reçoit un numéro calculé)
Pas gravé (jamais appelé) : AvancerAirs, AttendreAcces, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireTexte,
EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, EffacerCarte, Hasard, EffacerFond,
EffacerPanneau, Notes
Fonctions du langage appelées :
  sprite()       × 1    50 o sur place, dans glisse_croix()
  bouton()       × 4    20 o sur place, dans glisse_croix()  + LireManette (37 o)
  image()        × 1     6 o sur place, dans main()  + AttendreImage (28 o)
Tes fonctions appelées :
  glisse_croix() × 1    22 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <glisse_croix>   // un lutin qui suit la croix, au pixel près
#include <ALPHABET>       // les lettres de la police : ALPHABET[0] est le A
```

2. UNE VARIABLE — PX

```
uint8_t px = 20;      // le B, en pixels
```

3. UNE VARIABLE — PY

```
uint8_t py = 20;
```

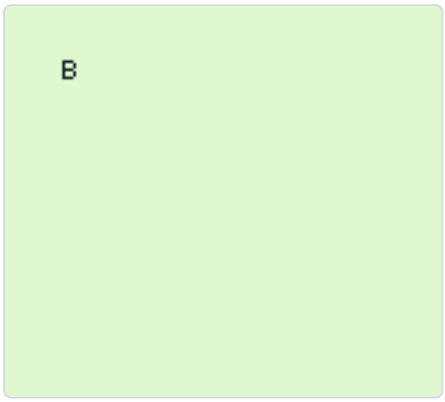
4. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
  while (true) {
    image();
    glisse_croix(0, px, py, ALPHABET[1], 3);
  }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.

la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

> Un B qui suit la croix, depuis le haut à gauche.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ 60 pixels en 20 images

Ensuite : **0.74.2. Glisser plus vite : glisse_croix à 3 — doublé, deux positions** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++